Trabajo Práctico — Sistema de Reconocimiento Facial

Tecnicatura Universitaria en Inteligencia Artificial — FCEIA (UNR)
Materia: Computer Vision

Modalidad	Grupal (hasta 2 integrantes)
Fecha de entrega	**03/11/2026**
Forma de entrega	Pull Request **sin mergear** hacia `main` en el repositorio propio del grupo
Repositorio base	https://github.com/EzequielMatiasArevalo/tuia-face-recognition-app

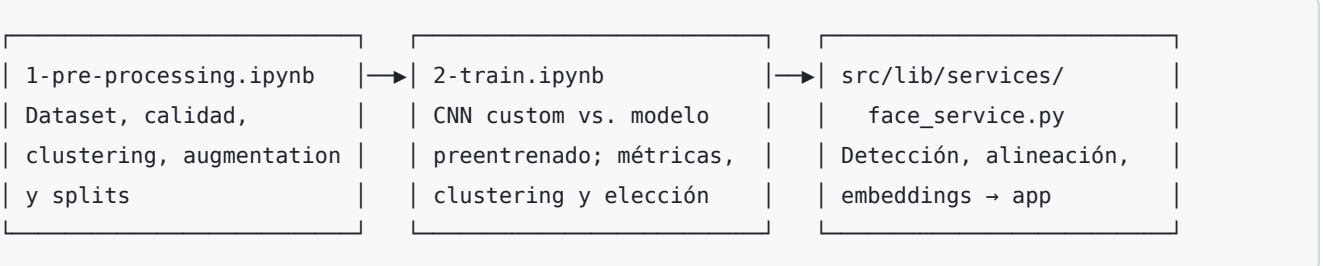
Índice

0. Objetivo general
1. Paso 0 — Preparar el repositorio (fork + feature branch)
2. Parte 1 — Dataset y preprocesamiento (`1-pre-processing.ipynb`)
3. Parte 2 — Entrenamiento y comparación de modelos (`2-train.ipynb`)
4. Parte 3 — Integración en la aplicación (`face_service.py`)
5. Entrega: Pull Request
6. Criterios de evaluación
7. Recomendaciones generales y errores frecuentes

0. Objetivo general

Construir de punta a punta un sistema de **reconocimiento facial**: desde la elección y curado de un dataset, pasando por el entrenamiento y la comparación de modelos, hasta la integración del modelo elegido en una aplicación web funcional que permita **registrar identidades** y **reconocer personas** en imágenes nuevas.

El trabajo se divide en tres partes, y cada una construye sobre la anterior:



Más allá de "que funcione", **se evalúa el criterio**. Esperamos que cada decisión (dataset, preprocesamiento, arquitectura, umbral, métrica) esté **justificada con evidencia**: gráficos, tablas, números o referencias.

1. Paso 0 — Preparar el repositorio

1.1 Fork del repositorio

1. Un integrante del grupo hace **fork** del repositorio base:
   https://github.com/EzequielMatiasArevalo/tuia-face-recognition-app
2. Agrega al resto de los integrantes como colaboradores del fork (*Settings* → *Collaborators*).
3. Clonen **su fork** (no el repositorio de la cátedra):

```
git clone git@github.com:<usuario-o-org>/tuia-face-recognition-app.git
cd tuia-face-recognition-app
```

> **¿Repositorio privado?** GitHub no permite cambiar a privado un fork de un repositorio público. Si el grupo prefiere un repositorio privado, en lugar de hacer fork **dupliquen** el repositorio:
>
> ```
> git clone --bare https://github.com/EzequielMatiasArevalo/tuia-face-recognition-app.git
> cd tuia-face-recognition-app.git
> git push --mirror git@github.com:<usuario-o-org>/<repo-privado>.git
> cd .. && rm -rf tuia-face-recognition-app.git
> ```
>
> Si el repositorio es **privado**, es **obligatorio** agregar a los docentes de la cátedra con permisos de **Maintain** (si el repo está en una organización) o como **colaboradores** (si está en una cuenta personal), para que puedan ver y revisar el PR. Un repositorio privado al que los docentes no tengan acceso se considera **NO ENTREGADO**.

1.2 Feature branch

Todo el trabajo se hace en una rama propia, **nunca directamente sobre** `main`:

```
git checkout -b feature/tp-reconocimiento-facial
git push -u origin feature/tp-reconocimiento-facial
```

Recomendaciones:

- Hagan commits chicos y con mensajes descriptivos (`feat: implementar detect_faces con SCRFD`, `docs: análisis de resoluciones del dataset`). El historial también se mira.
- Pueden abrir el Pull Request en modo **Draft** desde el principio e ir pusheando. Al final lo marcan como *Ready for review*.
- `main` debe quedar igual al repositorio base: es la referencia contra la que se revisa el diff.

1.3 Levantar el entorno

Sigan las instrucciones del README.md (opción Docker u opción local con `uv`). Antes de escribir código, verifiquen que:

- El frontend abre en `http://localhost:8080`.
- La documentación interactiva del backend abre en `http://localhost:8000/docs` (Swagger, útil para probar endpoints sin el frontend).
- PostgreSQL + pgvector levanta (`docker compose up postgres -d`).

> El backend **no va a arrancar** hasta que exista un archivo de modelo en `models/` con el nombre configurado en `MODEL_NAME` (ver `_load_model` en `face_service.py` y el endpoint `/health`). Es esperable en este punto.

---

2. Parte 1 — Dataset y preprocesamiento

Archivo: `1-pre-processing.ipynb` (o en Google Colab, ver 5.2)

Todo el tratamiento del dataset se hace y se documenta en esta notebook. Tiene que poder leerse como un informe: celdas Markdown que explican qué se hace y por qué, código, y salidas (gráficos y tablas) **ejecutadas y guardadas**. Una notebook sin salidas visibles se considera **NO ENTREGADA**.

2.1 Elección del dataset

Pueden usar un dataset público, uno propio, o una combinación. Algunas opciones:

Dataset	Identidades / imágenes	Comentarios
LFW (Labeled Faces in the Wild)	~5.700 / ~13.000	Clásico para verificación (tiene protocolo de pares oficial). Muy desbalanceado: la mayoría de las personas tiene 1 sola imagen. Disponible en `sklearn.datasets.fetch_lfw_people` / `fetch_lfw_pairs`.
Pins Face Recognition (Kaggle)	~105 / ~17.000	Celebridades con muchas imágenes por persona. Cómodo para entrenar una CNN de clasificación.
CelebA (con etiquetas de identidad)	~10.000 / ~200.000	Gran variedad. Pesado: conviene usar un subconjunto.
CASIA-WebFace	~10.000 / ~490.000	Dataset de entrenamiento estándar para modelos de reconocimiento facial. Ruido en las etiquetas.
FaceScrub, **VGGFace2**, **DigiFace-1M** (sintético)	Varía	Verifiquen disponibilidad y licencia antes de elegirlo.
Dataset propio	—	Fotos del grupo, amigos o familia **con consentimiento explícito**. Ver sección 2.8.

Se espera:

- Justificar la elección: cantidad de identidades, imágenes por identidad, variabilidad (pose, iluminación, edad, oclusiones), licencia de uso y relación con el caso de uso de la app.
- Describir explícitamente las **limitaciones** del dataset (sesgos demográficos, calidad, tipo de imágenes) y cómo pueden afectar los resultados.
- Si combinan datasets, explicar cómo unifican el formato y cómo evitan solapamiento de identidades.

> **Sugerencia:** filtren identidades con un mínimo de imágenes (por ejemplo, ≥ 10–20 por persona) para entrenar la CNN custom. Con 1 o 2 imágenes por clase una CNN desde cero no aprende nada útil.

2.2 Análisis exploratorio (EDA)

Como mínimo:

- **Cantidad de imágenes por identidad**: histograma y estadísticas (media, mediana, mínimo, máximo). Discutir el **desbalance de clases**.
- **Resoluciones**: distribución de ancho × alto, relación de aspecto y tamaño del rostro dentro de la imagen. Identificar imágenes demasiado chicas para el `FACE_SIZE` objetivo (112×112 por defecto).
- **Canales y formato**: RGB / escala de grises, extensiones, imágenes corruptas o que no abren.
- **Muestra visual**: grilla con ejemplos por identidad, incluyendo casos "difíciles".

- **Variaciones**: iluminación (histograma de brillo / contraste), pose, expresión y oclusiones (anteojos, barbijo). Al menos una parte puede ser cualitativa.

2.3 Control de calidad y limpieza

Definan **reglas de filtrado explícitas y reproducibles** y reporten cuántas imágenes descarta cada una:

- Imágenes donde el detector **no encuentra un rostro** o encuentra **más de uno** (en entrenamiento conviene quedarse con imágenes de un solo rostro, o con el rostro principal).
- Rostros con resolución menor a un mínimo (por ejemplo, bbox < 40 px de lado).
- Imágenes **borrosas** (por ejemplo, varianza del Laplaciano bajo un umbral) o con exposición extrema.
- **Duplicados y casi-duplicados** (hash perceptual, o similitud de embeddings > 0.95).
- Archivos corruptos.

Muestren ejemplos de imágenes descartadas por cada regla: así se valida que el umbral elegido tiene sentido.

2.4 Detección y alineación (normalización)

El modelo debe recibir rostros **recortados y alineados de forma consistente**. Se espera que:

- Elijan un detector (SCRFD/RetinaFace de InsightFace, YuNet de OpenCV, MTCNN, etc.) y justifiquen la elección.
- Alineen usando los **5 landmarks** (ojos, nariz, comisuras de la boca) con una transformación de similitud hacia una plantilla de referencia (por ejemplo, `insightface.utils.face_align.norm_crop`), y redimensionen a `FACE_SIZE`.
- Muestren un **antes / después** de la alineación.

> ⚠️ **Muy importante:** el **mismo pipeline** de detección, alineación y normalización (tamaño, orden de canales BGR/RGB, escala de píxeles, media/desvío) debe usarse en la notebook de entrenamiento **y** en `face_service.py`. Es la causa número uno de que "el modelo anda en la notebook pero no en la app". Se recomienda encapsularlo en funciones reutilizables.

2.5 Clustering exploratorio

Usen clustering como herramienta de **análisis del dataset**, no solo como gráfico decorativo:

- Extraigan embeddings con un modelo preentrenado (o features más simples) sobre las imágenes curadas.
- Proyecten en 2D con **t-SNE** (disponible en `scikit-learn`) o **UMAP**, coloreando por identidad.
- Apliquen un algoritmo de clustering (K-Means, DBSCAN, aglomerativo) y comparen los clusters con las etiquetas reales (ARI, NMI, *silhouette*).
- **Interpreten:** ¿qué identidades se mezclan? ¿Hay imágenes mal etiquetadas u *outliers* (imagen lejos de su propia clase)? ¿Hay sub-clusters por condiciones de captura (por ejemplo, blanco y negro o perfil)? Si encuentran errores de etiquetado, documenten si los corrigen o los descartan.

2.6 Data augmentation

`albumentations` ya está en `requirements.txt`.

- Definan un pipeline de augmentation **acorde al dominio facial**: flip horizontal, pequeñas rotaciones/traslaciones, *color jitter*, brillo/contraste, *blur*, ruido, *coarse dropout* (simula oclusiones), compresión JPEG.
- **Eviten transformaciones que destruyan la identidad o que no ocurren en la realidad** (flips verticales, rotaciones de 90°, distorsiones geométricas fuertes). Justifiquen cada transformación incluida o excluida.
- Muestren una grilla de ejemplos aumentados de una misma imagen.
- La augmentation se aplica **solo sobre el conjunto de entrenamiento**, nunca sobre validación ni test.
- Si la usan para mitigar el desbalance, explíquenlo y compárenlo con otras alternativas (*weighted sampling*, *class weights*).

2.7 Partición de datos (splits)

- Definan **train / validation / test** y justifiquen proporciones y estrategia.
- Distingan los dos escenarios del problema:
  - **Closed-set (identificación):** las mismas identidades en train y test, con imágenes distintas. Split estratificado por identidad.
  - **Open-set (verificación):** identidades de test **que el modelo nunca vio**. Es el escenario real de la app, ya que se registran personas nuevas. Split **por identidad**.
- Verifiquen que no haya **fuga de datos** (la misma imagen o un casi-duplicado en train y test).
- Fijen semillas y guarden los splits (CSV/JSON con rutas y etiquetas) para que la notebook 2 los reutilice.

2.8 Ética, privacidad y licencias

Los rostros son **datos biométricos**, una categoría de dato sensible. Incluyan una sección breve que contemple:

- Licencia del dataset y si permite el uso que le dan.
- Si usan fotos propias: **consentimiento informado** de cada persona. No suban imágenes de terceros al repositorio (el `.gitignore` ya excluye `data/*`).
- Sesgos del dataset (género, edad, tono de piel) y su posible impacto en el desempeño.
- Referencia a la Ley 25.326 de Protección de Datos Personales (Argentina) y a los riesgos de uso de reconocimiento facial.

2.9 Salidas esperadas de la notebook 1

- ▢ Justificación del dataset y descripción de sus limitaciones.
- ▢ EDA: distribución de clases, resoluciones, ejemplos visuales y variaciones.
- ▢ Reglas de calidad con cantidad de imágenes descartadas por regla y ejemplos.
- ▢ Pipeline de detección + alineación con visualización antes/después.
- ▢ Clustering exploratorio con interpretación.
- ▢ Pipeline de augmentation justificado y visualizado.

- [ ] Splits guardados y verificación de ausencia de fuga de datos.
- [ ] Tabla resumen final: cantidad de identidades e imágenes por split.
- [ ] Sección de ética y licencias.

---

3. Parte 2 — Entrenamiento y comparación de modelos

Archivo: `2-train.ipynb` (o en Google Colab, ver 5.2)

Hay que comparar **al menos dos** enfoques sobre **el mismo conjunto de test** y con **las mismas métricas**:

	Modelo A — CNN Custom	**Modelo B — Modelo preentrenado**
Qué es	Una CNN diseñada y entrenada por el grupo, desde cero o con un backbone liviano, sobre el dataset de la Parte 1	Un modelo de estado del arte, por ejemplo **InsightFace (ArcFace, `buffalo_l` / `buffalo_s`)**, FaceNet o similar
Objetivo	Entender qué se puede lograr con un modelo propio y datos limitados	Línea de base fuerte y de referencia

3.1 Modelo A — CNN Custom

El objetivo final **no es clasificar** las identidades de entrenamiento, sino generar **embeddings** (vectores) donde las caras de una misma persona queden cerca y las de personas distintas queden lejos. Así la app puede reconocer personas registradas después del entrenamiento.

Enfoques aceptados:

1. **Clasificación + embedding:** entrenar con softmax / cross-entropy sobre N identidades y usar la **capa previa al clasificador** (normalizada L2) como embedding. Es el más simple para arrancar.
2. **Metric learning:** *Triplet loss*, *Contrastive loss* o **márgenes angulares** (ArcFace / CosFace / SphereFace). Más complejo, pero produce embeddings de mayor calidad.

Se espera documentar:

- Arquitectura: capas, cantidad de parámetros, dimensión del embedding y diagrama o `summary`. Justificar decisiones como profundidad, BatchNorm, Dropout y *global pooling*.
- Hiperparámetros: optimizador, *learning rate* y *scheduler*, *batch size*, épocas, regularización.
- **Curvas de entrenamiento** (loss y accuracy de train vs. validation) y discusión de *overfitting* / *underfitting*.
- Experimentos realizados: qué probaron, qué no funcionó y por qué. Una tabla de experimentos es suficiente.

> **Sugerencias:** arranquen con algo chico (4–6 bloques conv) y entrada de 112×112. Usen *early stopping* y guarden el mejor checkpoint según validación. Si no tienen GPU, usen Google Colab o Kaggle Notebooks.

> **Opcional:** un *fine-tuning* de un backbone preentrenado (ResNet18, MobileNet, EfficientNet vía `timm` o `torchvision`) como tercer modelo de comparación.

3.2 Modelo B — Modelo preentrenado (ej. InsightFace)

- Usen el modelo como **extractor de embeddings** sobre su dataset, sin reentrenar (o con *fine-tuning* opcional).
- Documenten qué modelo usan (detector y reconocedor), su arquitectura, con qué datos fue entrenado, la dimensión del embedding (512 en ArcFace) y su **licencia** (los modelos preentrenados de InsightFace son para uso **no comercial / investigación**).
- Usen exactamente el mismo split de test que el Modelo A.

3.3 Evaluación

Ambos modelos se evalúan sobre los **embeddings**, con el mismo protocolo:

a) Verificación (¿son la misma persona?)

- Generen pares positivos y negativos balanceados a partir del test (o usen el protocolo de pares de LFW).
- Calculen la similitud coseno de cada par y reporten **curva ROC, AUC, EER** y **TAR @ FAR** (por ejemplo, FAR = 1e-2 y 1e-3).
- Grafiquen el **histograma de similitudes** de pares positivos vs. negativos. Visualmente deberían separarse.

b) Identificación (¿quién es?)

- Armen una **galería** (pocas imágenes registradas por identidad, imitando el `/insert` de la app) y un conjunto de **consultas** (*probes*).
- Reporten **accuracy Top-1 (y Top-5)**, **precision, recall y F1** (macro) y la **matriz de confusión** (o las identidades más confundidas si son muchas).
- Incluyan identidades **desconocidas** (no registradas en la galería) y midan cuántas se rechazan correctamente como `unknown`.

c) Elección del umbral

- La app usa `SIMILARITY_THRESHOLD` para decidir `unknown`. **Calibren el umbral con datos de validación**, no a ojo: por ejemplo, el punto EER o el umbral que da un FAR objetivo. Grafiquen precision/recall (o FAR/FRR) en función del umbral.
- El umbral elegido es el que van a configurar en el `.env` de la app.

d) Clustering de embeddings

- Proyección t-SNE/UMAP de los embeddings de **test** de cada modelo, lado a lado y coloreada por identidad.
- Métricas de clustering (silhouette, ARI/NMI con K-Means o aglomerativo) para cuantificar qué modelo separa mejor las identidades.

e) Análisis de errores

- Muestren ejemplos de **falsos positivos** (personas distintas aceptadas como la misma) y **falsos negativos** (la misma persona rechazada), con las imágenes y su score.
- Formulen hipótesis sobre las causas (pose, iluminación, baja resolución, parecido real, etiqueta errónea).

f) Costo computacional

- Tamaño del modelo (MB, parámetros), tiempo de inferencia por rostro en CPU y dimensión del embedding.

3.4 Comparación y elección final

Cierren la notebook con:

- Una **tabla comparativa** de todas las métricas anteriores para ambos modelos.
- Una **conclusión justificada** de qué modelo eligen para la app, considerando desempeño, costo, licencia y facilidad de integración. Es válido elegir el preentrenado, pero el análisis de por qué la CNN custom rinde menos (o más) es parte central de la nota.

3.5 Exportación del modelo

El backend carga el modelo desde `models/<MODEL_NAME>` con `FaceService._load_model`, que admite:

- **`.pth`**: se carga con `torch.load(path, weights_only=False)`. Esto implica guardar el **modelo completo** (`torch.save(model, path)`), no solo el `state_dict`, y que **la clase del modelo sea importable** desde el backend con el mismo módulo y nombre que tenía al guardarse. Si definen la clase dentro de la notebook, el `torch.load` va a fallar en la app. Pónganla en un módulo (por ejemplo, `src/lib/models/custom_cnn.py`) e impórtenla desde la notebook.
- **`.onnx`**: se carga con `onnxruntime.InferenceSession`. **Es la opción recomendada**: no depende del código de la clase. Exporten con `torch.onnx.export`. Para InsightFace, los modelos ya vienen en `.onnx` (por ejemplo, `w600k_r50.onnx` para reconocimiento y `det_10g.onnx` para detección dentro del paquete `buffalo_l`).

Si necesitan cargar más de un archivo (detector + reconocedor) o cambiar la forma de carga, pueden modificar `_load_model` / `__init__`, **justificándolo en el PR**.

> Los archivos `*.pth` y `*.onnx` están en el `.gitignore` y **no deben subirse al repositorio**. Súbanlos a Google Drive, Hugging Face Hub u otro servicio con un **link público de solo lectura**, y pongan el link en el README y en el PR.

3.6 Salidas esperadas de la notebook 2

- ▢ Modelo A: arquitectura, hiperparámetros, curvas de entrenamiento y experimentos.
- ▢ Modelo B: descripción, licencia y extracción de embeddings.
- ▢ Verificación: ROC, AUC, EER, TAR@FAR e histogramas de similitud.
- ▢ Identificación: Top-1/Top-5, precision, recall, F1, matriz de confusión y manejo de desconocidos.
- ▢ Umbral calibrado con su justificación.
- ▢ Clustering de embeddings de ambos modelos, con visualización y métricas.

- [ ] Análisis de errores con ejemplos visuales.
- [ ] Costo computacional.
- [ ] Tabla comparativa final y elección justificada.
- [ ] Modelo exportado y link de descarga.

---

4. Parte 3 — Integración en la aplicación

Archivo principal: `src/lib/services/face_service.py`

4.1 Cómo funciona la app

```
Frontend (Gradio :8080)
   │  POST /upload  → guarda la imagen y devuelve su path
   │  POST /insert  → {identity, image_path}   ─┐
   │  POST /predict → {source_path}             ├─▶ HTTP 202 + job_id (procesamiento asincrónico)
   │  GET  /status/{job_id} → done | inProgress | failed + links al resultado
   ▼
Backend (FastAPI :8000) ── FaceService ──▶ EmbeddingStore (PostgreSQL + pgvector, o JSON)
```

- **Registrar identidad** (`register_identity`): `_load_image` → `detect_faces` (debe haber **exactamente 1** rostro) → `align_face` → `extract_embedding_from_face` → se guarda el embedding en la base y el recorte alineado en `output/` .
- **Predecir** (`predict`): para **cada** rostro detectado se ejecuta `align_face` → `extract_embedding_from_face` → `identify` (busca el embedding más similar; si el score queda por debajo de `SIMILARITY_THRESHOLD` , devuelve `unknown`). El resultado se escribe como JSON y el frontend dibuja cajas, etiquetas y keypoints.

Lo que ya está resuelto y **no hace falta tocar**: la API, el manejo de jobs asincrónicos, la persistencia (pgvector / JSON), la similitud (`cosine` / `l2`), `identify` , `register_identity` , `predict` y el frontend.

4.2 Funciones a implementar

Hoy las tres funciones lanzan `NotImplementedError` :

`detect_faces(self, image) -> list[tuple[int, int, int, int]]`

- **Entrada:** imagen `np.ndarray` en **BGR uint8** (como la devuelve `cv2.imread`).
- **Salida:** lista de cajas `(x1, y1, x2, y2)` en **píxeles enteros** de la imagen original. Si no hay rostros, una lista vacía (no `None` ni excepción).
- **Recomendaciones:**
  - Usen el mismo detector que en la Parte 1.
  - Usen `self._clip_xyxy(...)` para que las cajas no se salgan de la imagen.
  - Filtren detecciones con score bajo o tamaño mínimo. Ese umbral también va al `.env` , no hardcodeado.
  - Inicialicen el detector **una sola vez** (en `__init__`), no en cada llamada.

`align_face(self, image, box) -> AlignedFace`

- **Entrada:** la imagen completa (BGR) y una caja `(x1, y1, x2, y2)`.
- **Salida:** un `AlignedFace` (ver `src/lib/schemas.py`) con:
  - `bbox`: la caja.
  - `image`: el rostro **alineado** de `self.face_size × self.face_size`, en BGR uint8. Se guarda con `cv2.imwrite` y el frontend lo muestra al registrar.
  - `keypoints`: array de forma `(5, 2)` con los landmarks **en coordenadas relativas a la esquina superior izquierda de la caja** (`x - x1`, `y - y1`), porque el frontend les vuelve a sumar `(x1, y1)` para dibujarlos. Orden esperado: `k0` ojo izquierdo, `k1` ojo derecho, `k2` nariz, `k3` comisura izquierda, `k4` comisura derecha. Si su detector no da landmarks, pueden devolver `None` (el frontend dibuja solo la caja), pero se valora tenerlos.
  - `embedding` (opcional): si su pipeline calcula el embedding junto con la alineación (por ejemplo, `FaceAnalysis` de InsightFace), pueden guardarlo acá y reutilizarlo en el paso siguiente.
- **Recomendaciones:** apliquen **la misma alineación** que en el entrenamiento. Para obtener landmarks precisos, conviene detectarlos en la imagen completa o en un recorte con margen alrededor de la caja, no en el recorte justo.

`extract_embedding_from_face(self, face: AlignedFace) -> list[float]`

- **Entrada:** el `AlignedFace` del paso anterior.
- **Salida:** `list[float]` de longitud **exactamente** `EMBEDDING_DIM` (se configura en `.env`; la tabla de pgvector se crea como `vector(EMBEDDING_DIM)`).
- **Recomendaciones:**
  - Repliquen el preprocesamiento del entrenamiento: BGR→RGB si corresponde, escala `[0,1]` o `(x-127.5)/127.5`, normalización por media/desvío, `HWC→CHW` y dimensión de batch.
  - Con `.pth`: `model.eval()` y `torch.no_grad()`. Con `.onnx`: `self.model.run(None, {input_name: tensor})`.
  - **Normalicen L2** el embedding antes de devolverlo.
  - Devuelvan floats de Python (`emb.astype(float).tolist()`), no tensores ni `np.float32`.

4.3 Configuración

- Copien el modelo a `models/` y actualicen `MODEL_NAME`, `EMBEDDING_DIM`, `FACE_SIZE`, `SIMILARITY_METRIC` y `SIMILARITY_THRESHOLD` (el umbral calibrado en la Parte 2) en `.env.docker.example` y `.env.local.example`.
- **No hardcodeen parámetros** en el código: todo lo configurable va por `.env` y `src/lib/config.py`.
- Si cambian `EMBEDDING_DIM` y ya existía la tabla, el store la recrea (y **borra** los registros previos). Es esperable.
- Si agregan dependencias, agréguenlas a `requirements.txt` con versión fija y reconstruyan la imagen (`docker compose build`).

4.4 Validación end-to-end

Demuestren que la app funciona:

1. Registrar al menos **3–5 identidades** desde el frontend (pestaña *Registrar identidad*), idealmente con más de una imagen por persona.
2. Predecir sobre imágenes **no usadas en el registro**: con una sola persona, con varias personas y con alguien **no registrado** (debe dar `unknown`).
3. Incluir en el PR **capturas de pantalla** del frontend mostrando cajas, keypoints y etiquetas.
4. Probar casos borde: imagen sin rostros (predict devuelve una lista vacía), registro con 2 rostros (debe fallar con un mensaje claro en `/status`).

4.5 Mejoras opcionales (suman puntos)

- Usar `PgVectorEmbeddingStore.search` (búsqueda por similitud en la base con el operador `<=>`) en lugar de recorrer todos los registros en Python.
- Soporte de **video** (`source_type="video"` ya existe en el schema): procesar frames muestreados.
- Agregación de varias imágenes por identidad (promedio de embeddings).
- Tests unitarios de las funciones implementadas.
- Actualizar el `README.md`: completar las secciones *Modelo y fine-tuning* y *Dataset* y corregir el pipeline descripto según su implementación.

---

5. Entrega: Pull Request

La entrega es un **Pull Request desde `feature/...` hacia `main` de su propio repositorio**, **SIN MERGEAR**. El PR se va a revisar y comentar sobre el diff. No abran el PR contra el repositorio de la cátedra.

5.1 Contenido del PR

El diff debe incluir:

- `1-pre-processing.ipynb` y `2-train.ipynb` (ver 5.2). Completen los nombres de los integrantes en la celda *Equipo*.
- `src/lib/services/face_service.py` con las tres funciones implementadas, más cualquier módulo auxiliar que hayan agregado.
- Archivos `.env.*.example` actualizados.
- `requirements.txt` actualizado (si corresponde).
- `README.md` actualizado.

No debe incluir: datasets, pesos de modelos (`.pth` / `.onnx`), archivos `.env` con secretos, ni `output/`.

5.2 Notebooks: repositorio o Google Colab

Las notebooks se pueden entregar de dos formas:

1. **Commiteadas en el repositorio**, reemplazando `1-pre-processing.ipynb` y `2-train.ipynb`.
2. **En Google Colab.** En este caso, las notebooks `1-pre-processing.ipynb` y `2-train.ipynb` del repositorio igual deben commitearse y funcionan como **notebooks de referencia**. Cada una debe contener, en una celda Markdown al inicio, el **link a la notebook de Colab**

correspondiente. Ese link debe tener **permiso de lectura para los docentes**: compártanlo con sus cuentas o con "Cualquier persona con el enlace" como *Lector*.

En **ambos casos**, las notebooks deben entregarse **ejecutadas y con la salida del código visible**: gráficos, tablas, métricas y logs de entrenamiento.

> ⚠️ **Se considera NO ENTREGADO:**
>
> - Una notebook sin ejecutar o sin las salidas visibles (por ejemplo, guardada con "*Clear all outputs*" o con la opción de Colab que omite las salidas al guardar).
> - Una notebook en Colab cuyo link no figure en la notebook de referencia del repositorio.
> - Una notebook en Colab a la que los docentes no tengan permiso de lectura.

Recomendaciones si usan Colab:

- Verifiquen el link desde una ventana de incógnito o con una cuenta ajena al grupo.
- No modifiquen la notebook de Colab después de la fecha de entrega. Como resguardo, también pueden descargarla (*Archivo* → *Descargar* → *.ipynb*) y commitearla en el repositorio.
- Guarden los datos y los modelos en Google Drive y documenten en la notebook cómo montarlos y de dónde descargarlos, para que la notebook sea reproducible.

5.3 Descripción del PR

Usen esta plantilla:

```
## Integrantes
- Nombre Apellido — legajo — @usuario-github
- Nombre Apellido — legajo — @usuario-github

## Resumen
Breve descripción del trabajo realizado.

## Dataset
Nombre, fuente, cantidad de identidades/imágenes final y link de descarga (si es propio o
procesado).

## Modelos comparados
| Métrica | CNN Custom | <Modelo preentrenado> |
|---|---|---|
| AUC (verificación) | | |
| EER | | |
| TAR @ FAR=1e-3 | | |
| Top-1 accuracy | | |
| Precision / Recall / F1 (macro) | | |
| Tiempo inferencia CPU (ms/rostro) | | |
| Tamaño (MB) | | |

**Modelo elegido:** ... — **Justificación:** ...
```

```
**Umbral configurado:** ... (métrica: cosine / l2)

## Modelo
Link público de solo lectura: <link>
Ubicación esperada: `models/<MODEL_NAME>`

## Cómo correrlo
Pasos exactos para reproducir: descargar el modelo, configurar `.env` y `docker compose up`.
```

5.4 Checklist antes de entregar

- ☐ El PR apunta a `main` **de su repositorio** y **no está mergeado**.
- ☐ Si el repo es privado, los docentes tienen acceso (Maintain / colaborador).
- ☐ Las notebooks están **ejecutadas y con las salidas visibles**, en GitHub o en Colab.
- ☐ Si usaron Colab: las notebooks de referencia del repo tienen el link y los docentes pueden abrirlo (probado en incógnito).
- ☐ El link al modelo funciona desde una ventana de incógnito.
- ☐ Partiendo de un clon limpio, siguiendo **solo** las instrucciones del PR, `docker compose up` levanta y `/health` responde `ok`.
- ☐ No quedan `NotImplementedError` en `face_service.py`.

---

6. Criterios de evaluación

Se evalúan los siguientes aspectos:

- **Parte 1 — Dataset:** elección, EDA, calidad, clustering, augmentation, splits y ética.
- **Parte 2 — Modelos:** CNN custom, modelo preentrenado, evaluación rigurosa, clustering, análisis de errores y conclusión.
- **Parte 3 — App:** funcionamiento end-to-end con el modelo elegido y consistencia del pipeline.
- **Calidad de la entrega:** PR, documentación, reproducibilidad, código limpio e historial de commits.

Condiciones para aprobar:

- La app debe **registrar y predecir** con el modelo del grupo.
- Ambas notebooks deben estar ejecutadas y documentadas.
- El PR debe estar abierto, sin mergear y accesible para los docentes.

---

7. Recomendaciones generales y errores frecuentes

Organización

- Empiecen por el *pipeline* completo con algo simple (por ejemplo, InsightFace en la app) y después mejoren cada parte. Tener la app funcionando temprano les da un entorno para probar.

- Muevan el código reutilizable (detección, alineación, preprocesamiento, modelo) a módulos `.py` e impórtenlos desde las notebooks y desde el backend. Así evitan duplicar lógica y cometer inconsistencias.
- Fijen semillas (`random`, `numpy`, `torch`) para que los resultados sean reproducibles.

Errores frecuentes

- **BGR vs. RGB:** OpenCV lee en BGR; PIL, torchvision y la mayoría de los modelos entrenados en PyTorch esperan RGB.
- **Normalización distinta** entre entrenamiento e inferencia.
- **Evaluar con identidades de entrenamiento** y concluir que el modelo generaliza. Para verificación, el test debe tener identidades no vistas.
- **Augmentation sobre validación/test.**
- **Umbral elegido a ojo** o calibrado sobre el mismo conjunto en el que se reportan las métricas.
- **`torch.save(model.state_dict())`** cuando el backend espera el modelo completo, o clase no importable al hacer `torch.load`. Prefieran ONNX.
- **Dimensión del embedding distinta de `EMBEDDING_DIM`**: pgvector rechaza el insert.
- **Keypoints en coordenadas absolutas** en lugar de relativas a la caja: aparecen desplazados en el frontend.
- **Cargar el modelo o el detector en cada request:** la app se vuelve muy lenta.
- **Subir datasets o pesos al repositorio.**

Recursos útiles

- InsightFace: https://github.com/deepinsight/insightface (ver `python-package/`)
- ArcFace (Deng et al., 2019): https://arxiv.org/abs/1801.07698
- FaceNet / Triplet loss (Schroff et al., 2015): https://arxiv.org/abs/1503.03832
- LFW: https://vis-www.cs.umass.edu/lfw/
- Albumentations: https://albumentations.ai/docs/
- Exportar a ONNX: https://pytorch.org/docs/stable/onnx.html
- pgvector: https://github.com/pgvector/pgvector

Ante dudas, consulten por los canales de la cátedra. **¡Éxitos!**